



Politechnika Krakowska
Wydział Informatyki i Telekomunikacji

Studia Niestacjonarne

Sprawozdanie z przedmiotu:

Programowanie Równoległe i Rozproszone

Temat Projektu:

Detekcja Ruchu w Wideo

Wykonał:

Miłosz Gronowski

Spis treści

1.	Wstęp	3
1.1	Cel projektu	3
1.2	Zakres projektu	3
1.3	Podział pracy	3
2.	Opis problemu	5
2.1	Detekcja ruchu metodą różnicowania klatek	5
2.2	Analiza modelu tła i decyzja projektowa.....	5
2.3	Przetwarzanie morfologiczne w filtracji szumów	6
3.	Środowisko i konfiguracja sprzętowa.....	7
3.1	Środowisko programistyczne	7
3.2	Konfiguracja sprzętowa	7
4.	Implementacja	9
4.1	Optymalizacja i test modeli tła.....	9
4.2	Analiza ograniczeń algorytmu na podstawie wyników emiprycznych	11
4.3	Implementacja sekwencyjna (referencyjna)	13
4.4	Implementacja OpenMP	15
4.5	Implementacja MPI	16
4.6	Implementacja CUDA	18
5.	Metodologia testów wydajności.....	21
5.1	Dane Testowe	21
5.2	Wybór analizowanych plików	24
5.3	Parametry wersji programów.....	24
5.4	Parametry algorytmu	25
5.5	Przebieg testów	26
6.	Wyniki	27
6.1	Analiza wydajności dla testu długiego (traffic.mp4)	27
6.2	Analiza wydajności dla testu krótkiego (ball.mp4).....	29
6.4	Podsumowanie wyników i najważniejsze wnioski	33
7.	Instrukcja obsługi i uruchomienia	34
7.1	Wymagania.....	34
7.2	Uruchomienie programu.....	34
8.	Podsumowanie projektu	36

1. Wstęp

1.1 Cel projektu

Celem projektu było zaprojektowanie, implementacja oraz analiza wydajnościowa zrównoleglonego systemu detekcji ruchu w wideo. Rdzeniem tego systemu jest klasyczny algorytm wizji komputerowej oparty na analizie różnicowej klatek obrazu, uzupełniony o operacje morfologiczne służące do filtracji zakłóceń.

Główny nacisk położono na analizę, jak paradygmaty programowania równoległego i rozproszonego wpływają na przepustowość przetwarzania potoku wideo (wyrażoną w liczbie klatek na sekundę – FPS) oraz na czas ich realizacji. Zaimplementowano trzy niezależne warianty optymalizacji – OpenMP, MPI, CUDA za pomocą OpenCV.

Finalnym celem projektu, poza weryfikacją poprawnej detekcji wizualnej, było przeprowadzenie testów porównawczych w celu wyznaczenia krzywych przyspieszenia (S) oraz efektywności zrównoleglenia (E) dla różnych rozdzielczości wejściowych, co pozwoli wskazać granicę opłacalności stosowania poszczególnych technologii w rzeczywistych systemach analizy wideo. Porównano także metody pod kątem czasu (T) działania oraz przepustowości (FPS). Dla zróżnicowania analizowanego materiału, testy odbyły się na trzech plikach wideo o różnej rozdzielczości i długości trwania.

1.2 Zakres projektu

W ramach realizacji projektu zdefiniowano i wykonano następujący zakres prac inżynierskich i badawczych:

- **Wersja sekwencyjna** – stworzenie bazowego programu, który wykrywa ruch poprzez porównywanie kolejnych klatek filmu (Frame-to-Frame), wygładza obraz i czyści szумы.
- Automatyczne parametry – dodano kod, który sam zmienia wielkość filtrów i czułość w zależności od rozdzielczości badanego wideo.
- Wersja OpenMP – zrównoleglono operacje na pikselach na wiele wątków procesora przy użyciu biblioteki Numba.
- Wersja MPI – podzielono klatki filmu równo pomiędzy kilka niezależnych procesów, żeby każdy rdzeń procesora liczył swój kawałek wideo.
- Wersja CUDA – przeniesiono obliczenia na kartę graficzną za pomocą OpenCV Transparent API, wykorzystując rdzenie CUDA.
- Poprawa pomiarów – wypracowanie schematu pozwalającego na jednolite porównanie wersji programu np. poprzez metodykę badań.
- Testy – zmierzono czas działania, FPS, efektywność, przyspieszenia dla trzech różnych filmów o różnej specyfice i porównano wyniki pomiędzy metodami.

1.3 Podział pracy

CZYNNOŚĆ	MIŁOŚ GRONOWSKI
Bazowy program sekwencyjny i mechanizm dobierania parametrów	X
Logika wykrywania ruchu i ograniczenia	X

Opracowanie struktury katalogów	X
Wersje programów z użyciem OpenMP, MPI, CUDA	X
Przeprowadzenie testów wydajnościowych	X
Sporządzenie porównania i podsumowania wyników	X
Sporządzenie sprawozdania	X

2. Opis problemu

2.1 Detekcja ruchu metodą różnicowania klatek

Podstawowym mechanizmem wykrywania dynamiki w sekwencji wideo jest analiza zmian strumienia iluminacji w dziedzinie czasu. Metoda różnicowania klatek (ang. frame differencing) opiera się na założeniu, że ruch obiektu powoduje gwałtowną zmianę intensywności odpowiadających mu pikseli pomiędzy następującymi po sobie rejestracjami obrazu.

Dla cyfrowego sygnału wideo reprezentowanego jako trójwymiarowa macierz $I(x,y,t)$, gdzie (x,y) oznaczają współrzędne przestrzenne piksela, a t dyskretną oś czasu (numer klatki), obraz $D(x,y,t)$ w skali szarości wyznacza się poprzez obliczenie bezwzględnej wartości różnicy sygnałów:

$$D(x,y,t) = | I(x,y,t) - B(x,y,t) |$$

Gdzie $B(x,y,t)$ stanowi obraz referencyjny – w najprostszym podejściu jest to klatka poprzedzająca.

Stosowane jest progowanie, które zamienia obraz w maskę ruchu – piksel jest uznany za „ruszający się”, gdy różnica przekracza określony próg czułości. Formalny zapis:

$$M(x,y,t) = 255 \text{ dla } D(x,y,t) > r, \quad M(x,y,t) = 0 \text{ dla } D(x,y,t) \leq r,$$

To rozwiązanie niesie jednak za sobą potencjalne problemy, takie jak chociażby podatność na drgania rejestratora wideo, błyski, migotania lub zmiany oświetlenia, czy chociażby szum czujnika kamery.

2.2 Analiza modelu tła i decyzja projektowa

Zgodnie z założeniami projektu, system detekcji ruchu powinien opierać się na metodzie różnicowania klatek wspartej prostym modelem tła. W fazie projektowej przeanalizowano dwa warianty realizacji tego komponentu.

Wariant A: Model tła oparty na średniej kroczącej

W fazie projektowej przeanalizowano możliwość zastosowania adaptacyjnego modelu tła opartego na algorytmie średniej kroczącej (ang. Moving Average Background). Do teoretycznych zalet tego rozwiązania należy zdolność do ignorowania powolnych, naturalnych zmian w otoczeniu (np. pora dnia) oraz wysoka stabilność przy statycznym środowisku. Do kluczowych wad zalicza się jednak wysokie ryzyko rozmazywania konturów oraz podatność na powstawanie tzw. „efektu ducha” (pozostawianie powidoków w miejscach, które obiekt właśnie opuścił), wynikające z opóźnienia adaptacji matematycznej modelu. To właśnie ten problem napotkano przy implementacji tego rozwiązania, co skutkowało wybraniem bardziej optymalnej metody.

Wariant B: Model tła natychmiastowego

Mając na uwadze cel projektu, jakim jest optymalizacja potoku pod kątem przetwarzania równoległego i rozproszonego, jako prosty model tła zaimplementowano model tła natychmiastowego, gdzie obrazem referencyjnym tła jest klatka bezpośrednio poprzedzająca bieżącą. Wybór ten pozwolił na:

- **Likwidację zależności wideo** - porównanie klatki t z klatką $t-1$ jest operacją całkowicie lokalną w czasie. Pozwala to na swobodne rozpraszanie pakietów klatek do losowych węzłów obliczeniowych bez konieczności przesyłania historii wideo.

- **Optymalizację pamięci** - prosty model tła natychmiastowego operuje na pamięci podręcznej typu uint8, redukując narzut na pamięć.
- **Poprawę jakości** – przejście na ten model trwale zlikwidowało powstanie „efektu ducha”.

2.3 Przetwarzanie morfologiczne w filtracji szumów

Nawet po zastosowaniu binaryzacji obrazu różnicowego, uzyskana czarno-biała maska ruchu wciąż nie jest idealna i często zawiera drobne błędy i zakłócenia. W celu filtracji tego obrazu i scalenia poszarpanych detekcji w spójne obiekty, zastosowano operacje morfologiczne, które analizują otoczenie każdego piksela za pomocą matrycy testowej.

W projekcie czyszczenie maski ruchu opiera się na dwóch operacjach:

- **Otwarcie morfologiczne:** działa jak szybki filtr czyszczący, którego zadaniem jest usunięcie z obrazu małych, przypadkowych detekcji, które powstały w wyniku szumu, bez naruszania właściwych dynamicznych obszarów ruchu
- **Zamknięcie morfologiczne:** służy do wypełniania niewielkich dziur i szczelin wewnątrz wykrytych sylwetek oraz domykania ich kształtów, aby obiekty były pełne i spójne

Rezultatem jest czysta, wyraźna maska, na której zamiast chmury szumów widać spójne i pełne zarysy poruszających się obiektów, co umożliwia nałożenie czerwonych ramek (ang. bounding boxes).

3. Środowisko i konfiguracja sprzętowa

3.1 Środowisko programistyczne

Specyfikacja techniczna projektu:

- **Język programowania – Python** w wersji 3.12. Użyto go w ramach projektu do wszystkich wersji systemu ze względu na bogaty ekosystem bibliotek dedykowanych przetwarzaniu obrazów.
- **Środowisko programistyczne – Visual Studio Code.**
- **System:** Repozytorium projektu utrzymywano na **Git/GiHub**.
- **Biblioteki bazowe i pomocnicze:**
 - **opencv-python** – do wczytywania wideo, zapisu gotowych plików, konwersji kolorów oraz ramek na klatkach wynikowych.
 - **numpy** – do operacji na tablicach pikseli.
 - **time** – do mierzenia czasu działania pętli w celach porównawczych.
 - **os** – do zarządzania ścieżkami na dysku i tworzenia folderów wyjściowych.
- **Biblioteki i technologie zrównoleglające:**
 - **numba (njit, prange)** – do kompilacji JIT, realizowane w ramach wersji OpenMP.
 - **mpi4py** – do komunikacji międzyprocesowej w ramach wersji MPI.
 - **OpenCV Transparent API (T-API)** – wbudowana technologia akceleracji sprzętowej na GPU w OpenCV. Automatycznie deleguje ona procesy na pikselach bezpośrednio do rdzeni **CUDA** karty graficznej NVIDIA za pośrednictwem systemowych sterowników.

3.2 Konfiguracja sprzętowa

Wszystkie testy wydajnościowe, pomiary czasowe oraz renderowanie końcowych plików wideo zostały przeprowadzone na jednej, stałej maszynie testowej. Zapewniło to identyczne środowisko sprzętowe dla każdego eksperymentu, co jest kluczowym warunkiem do rzetelnego i bezpośredniego porównywania wskaźników takich jak czysty czas obliczeń czy osiągnięte przyspieszenie względne.

Specyfikacja techniczna:

- **Procesor główny (CPU)**
 - **Model** - AMD Ryzen 5 7535HS
 - **Taktowanie bazowe** – 3.30 GHz
 - **Rdzenie / wątki** – 6 rdzeni fizycznych / 12 wątków logicznych
- **Karta graficzna (GPU)**
 - **Model** - NVIDIA GeForce RTX 3050 Laptop GPU
 - **Pamięć VRAM** – 6.0 GB GDDR6
 - **Architektura** - Zgodna z technologią masowo równoległą NVIDIA CUDA

- **Pamięć operacyjna (RAM)**

- **Pojemność** – 16 GB DDR5

- **System operacyjny**

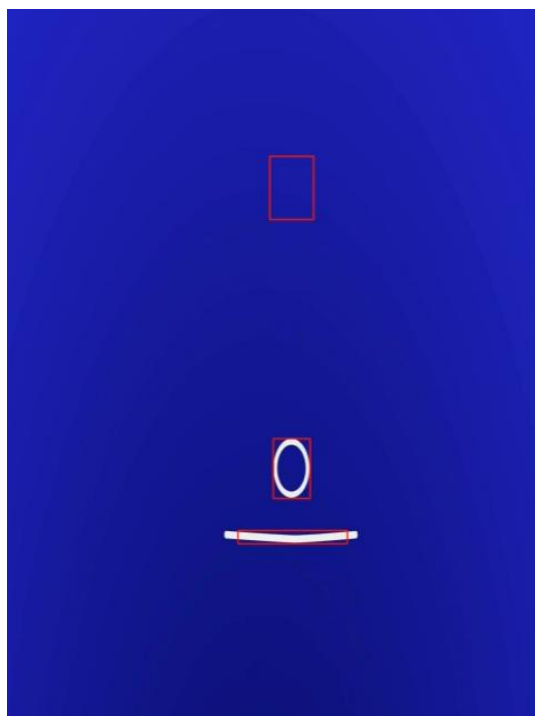
- **System** – Microsoft Windows 11 Home

4. Implementacja

4.1 Optymalizacja i test modeli tła

Przed przystąpieniem do implementacji algorytmu w środowiskach równoległych, przeprowadzono szczegółowe testy poprawności wizualnej w wersji sekwencyjnej (referencyjnej). Początkowa wersja podejścia testowego, opartego na powolnej akumulacji wagowej modelu tła oraz sztywnych parametrach filtrowania, ujawniło krytyczne problemy algorytmiczne:

- **Powstanie „efektu ducha”** – szybko poruszające się obiekty po opuszczeniu swojej dotychczasowej pozycji, pozostawiały na masce binarnej powidłoki (fałszywe ślady). System potrzebował kilkunastu klatek, aby zregenerować tło w odsłoniętym miejscu, generując w tym czasie puste, błędne ramki detekcji, co pokazuje przykład poniżej:

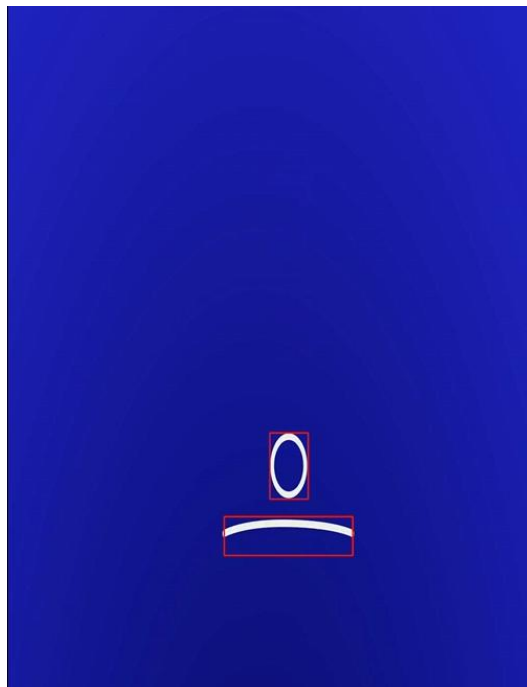


- **Masowe fałszywe alarmy dla wysokich rozdzielczości** – sztywne parametry (tj. próg binaryzacji, filtr Gaussa czy minimalna wartość konturu) sprawdzały się wyłącznie na plikach o niskiej rozdzielczości. Dla materiałów o wysokiej jakości, zwiększona gęstość pikseli oraz mikro-szumy matrycy wywoływały lawinowe rysowanie fałszywych ramek wokół drgających elementów tła, co pokazuje przykład poniżej:



W celu usunięcia tych wad i przygotowania stabilnej bazy, wprowadzono dwupoziomową optymalizację potoku danych:

- **Model różnicowania klatka-do-klatki (Frame-to-Frame)** – całkowicie wyeliminowano mechanizm długotrwałego uśredniania tła. Zamiast tego obraz wyznaczany jest bezpośrednio jako różnica intensywności między klatką bieżącą i poprzedzającą. Dzięki temu stara pozycja obiektu natychmiast znika z pamięci referencyjnej już w kolejnej klatce wideo, co zlikwidowało powstawanie „efektu ducha”. Dodatkowo, operacja ta ma charakter ściśle lokalny (brak konieczności pamiętania ułamkowej historii filmu), co drastycznie upraszcza strukturę danych. Przykład nagrania po zmianie:



- **Mechanizm automatycznego, adaptacyjnego skalowania filtrów** – wprowadzono moduł analizujący parametry techniczne wczytanego wideo. System dopasowuje siłę filtrów i progi detekcji do rozdzielczości przestrzennej klatki wedle ustalonych granic dla filmów wyższej i niższej jakości. Dzięki tym zmianom algorytm zachowuje stabilność, odcinając szum i fluktuację oświetlenia na dużych

matrycach, jednocześnie precyzyjnie śledząc kontury rzeczywistego ruchu. Przykład nagrania po zmianie:



Ważna uwaga metodologiczna: Wszystkie wypracowane i zoptymalizowane w tym kroku parametry (stałe matematyczne, rozmiary masek, warunki logiczne) zostały w niezmienionej formie przeniesione do kolejnych trzech programów równoległych (OpenMP, MPI, CUDA). Gwarantuje to pełną rzetelność testów wydajnościowych – każdy program wykonuje identyczną pracę na pikselach i generuje identyczny plik wynikowy wideo, a jedyną zmienną podlegającą badaniu jest architektura wykonawcza.

4.2 Analiza ograniczeń algorytmu na podstawie wyników emiprycznych

Podczas testów dostrajających potok detekcji zaobserwowano fundamentalne ograniczenie metody różnicowania klatek (Frame-to-Frame) w odniesieniu do złożonych obiektów jak np. człowiek w ruchu.

O ile w filmach o niższej jakości dopasowanie parametrów pozwalało na zabicie sylwetki w jeden blok, jak poniżej:



Tak w filmach o wyższej jakości następowało rozbicie na mniejsze bloki. Gdy obiekty o małej, geometrycznej i kontrastowej strukturze (np. piłka przemieszczająca się po murawie) były wykrywane bezbłędnie jako jednolite, zwarte bryły, o tyle sylwetka człowieka w ruchu wykazywała permanentną tendencję do rozpadania się na niezależne pod-kontury, co obrazuje stan potoku na rzucie poniżej:



Zjawisko to wynika bezpośrednio z ograniczeń samej metody różnicowania klatek:

- **Zbyt małe różnice między kolejnymi klatkami** – przez porównywanie kolejnych klatek po sobie, przesunięcie obiektu może być minimalne. W miejscach, gdzie ruch jest najwolniejszy lub obiekt ma jednolity kolor (np. spodnie osoby na rzucie ekranu), aparat w kolejnych klatkach widzi niemal identyczne piksele. Różnica matematyczna w tych miejscach wynosi zero, przez co algorytm uważa, że te fragmenty są w bezruchu i wycina je z maski ruchu, tworząc w niej czarne przerwy.

- **Próby połączenia tego morfologią** - próby „sklejania” przerw za pomocą większych filtrów morfologicznych oraz obniżania progu binaryzacji kończyły się niepowodzeniem. Gdy algorytm stawał

się wystarczająco czuły, by połączyć cały obiekt, natychmiast zaczynał zbierać masę szumów z tła, które zakłócały proces detekcji ruchu.

Dlaczego zostajemy przy tej metodzie?

Mimo tego problemu, model Frame-to-Frame z prostym tłem natychmiastowym jest optymalnym wyborem do realizacji celów tego projektu. Naszym priorytetem jest zrównoleglenie kodu i analiza wydajności. Ta metoda całkowicie likwiduje zależność danych w czasie, co upraszcza strukturę pamięci. Dzięki temu algorytm idealnie nadaje się do rozproszenia na wiele procesów i uruchomienia na rdzeniach GPU, co w zupełności wystarcza do poprawnego wyznaczenia krzywych przyspieszenia i efektywności systemu.

4.3 Implementacja sekwencyjna (referencyjna)

Wariant sekwencyjny stanowi bazę referencyjną (ang. baseline) dla całego projektu. Jego głównym zadaniem jest zapewnienie poprawnej detekcji wizualnej obiektów w ruchu oraz wyznaczenie bazowego czasu wykonania, względem którego wyliczane będą wskaźniki przyspieszenia i efektywności dla implementacji równoległych. Program realizuje przetwarzanie potokowe w architekturze synchronicznej, wykonując operacje sekwencyjnie dla każdej wczytanej klatki wewnątrz jednego wątku procesora. Przebieg algorytmu rozpoczyna się od pobrania parametrów technicznych strumienia wideo (szerokość, wysokość) i dynamicznego przypisania wag filtrów oraz progów czułości na podstawie najdłuższego boku klatki, co zapobiega tłumieniu ruchu w niskich rozdzielczościach oraz powstawaniu fałszywych detekcji w wysokich jakościach.

```
# parametry techniczne algorytmu zależne od rozdzielczości
max_side = max(width, height)

if max_side >= 1920:
    threshold_val = 20
    blur_kernel_size = (7, 7)
    morph_kernel_size = (7, 7)
    min_contour_area = 300
elif max_side >= 1280:
    threshold_val = 15
    blur_kernel_size = (5, 5)
    morph_kernel_size = (5, 5)
    min_contour_area = 150
else:
    threshold_val = 10
    blur_kernel_size = (3, 3)
    morph_kernel_size = (5, 5)
    min_contour_area = 50
```

Po ustaleniu środowiska wykonawczego program przechwytuje pierwszą klatkę filmu w celu zainicjalizowania prostego modelu tła natychmiastowego. Zostaje ona przekształcona do skali szarości oraz poddana wygładzeniu filtrem Gaussa, tworząc macierz referencyjną dla kolejnego kroku obliczeniowego. Następnie system uruchamia główną pętlę przetwarzania, w której każda nowo

wczytana klatka przechodzi identyczny preprocessing (konwersja do odcieni szarości i blur), po czym następuje kluczowa operacja wyznaczania matematycznej różnicy jasności pikseli za pomocą funkcji `cv2.absdiff` oraz jej progowanie. Uzyskana surowa maska ruchu jest natychmiast przekazywana do modułu filtracji morfologicznej, gdzie sekwencyjne wykonanie operacji otwarcia (ang. open) oraz zamknięcia (ang. close) pozwala na jednoczesne usunięcie drobnego szumu i scalenie rozbitych fragmentów obiektów.

```
# przetwarzanie wideo klatka po klatce
while True:
    ret, frame = cap.read()
    if not ret:
        break

    # preprocessing bieżącej klatki
    gray_curr = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    gray_curr = cv2.GaussianBlur(gray_curr, blur_kernel_size, 0)

    # różnicowanie klatek od modelu tła
    diff = cv2.absdiff(gray_curr, gray_prev)

    # progowanie
    _, mask = cv2.threshold(diff, threshold_val, 255, cv2.THRESH_BINARY)

    # asymetryczne czyszczenie maski w pętli
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, morph_kernel_size)
    opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel) # usuwanie szumu
    clean_mask = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel) # scalanie przerwanych konturów
```

Ostatnim etapem potoku wewnątrz pętli jest analiza geometryczna przestrzeni binarnej. Algorytm wyszukuje wektory brzegowe spójnych obszarów, odrzuca te, które nie spełniają minimalnego kryterium powierzchniowego (*min_contour_area*), a dla struktur poprawnie sklasyfikowanych jako obiekty ruchome wyznacza prostokąt opisujący (ang. bounding box), nanoszony w postaci czerwonej ramki na finalny obraz wideo. Gotowy obraz jest dodawany do listy w pamięci RAM w celu odseparowania operacji dyskowych od pomiaru czasu. Pętlę zamyka operacja przesunięcia rejestru, w której klatka bieżąca staje się klatką poprzednią dla następnej iteracji.

```
# wykrywanie konturów obiektów w ruchu i rysowanie ramek
contours, _ = cv2.findContours(clean_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
result_frame = frame.copy()

for contour in contours:
    if cv2.contourArea(contour) < min_contour_area:
        continue
    # wyznaczenie boxa dla poruszającego się obiektu
    (x, y, w, h) = cv2.boundingRect(contour)
    cv2.rectangle(result_frame, (x, y), (x + w, y + h), (0, 0, 255), 2)

# aktualizacja klatki - bieżąca będzie poprzednią
gray_prev = gray_curr.copy()
```

Dla każdego analizowanego materiału wideo tworzone jest podsumowanie w postaci informacji o rozdzielczości, ścieżce zapisu pliku w przeznaczonym do tego folderze, czystym czasie obliczeń (bez zapisu dyskowego), informacji o przetworzonych klatkach w pętli oraz wydajności przetwarzania.

Przedstawiona implementacja sekwencyjna stanowi bazę do tworzenia kolejnych implementacji z użyciem OpenMP, MPI oraz CUDA.

4.4 Implementacja OpenMP

Wariant równoległy korzystający z OpenMP opiera się dokładnie na tym samym schemacie co wersja sekwencyjna. Wszystko, co dotyczy obsługi plików – czyli otwieranie wideo, automatyczne dobieranie filtrów pod rozdzielczość oraz końcowy zapis gotowego filmu z czerwonymi ramkami – działa tu identycznie. Całą naszą uwagę skupiliśmy na kluczowych wymaganiach projektu: operacjach na pikselach, które w wersji sekwencyjnej były wykonywane po kolei, klatka po klatce także na progowaniu i postępie morfologii (operacjach otwarcia i zamknięcia), które w wersji sekwencyjnej obciążały jeden wątek procesora. W języku Python standardowy kod działa na jednym rdzeniu przez tzw. blokadę GIL (Global Interpreter Lock). Aby ją ominąć i zmusić komputer do prawdziwej pracy wielowątkowej, użyliśmy biblioteki Numba z funkcją kompilacji JIT (Just-In-Time). Dzięki temu Python w locie tłumaczy nasz kod na superszybkie instrukcje maszynowe, które potrafią działać na wielu rdzeniach procesora jednocześnie.

Głównym pomysłem na przyspieszenie programu było połączenie operacji analizy pikseli (rozmycia obrazu, odejmowania klatek od siebie oraz tworzenia czarno-białej maski ruchu) w jedną, wspólną funkcję o nazwie *openmp_pixel_pipeline*, po której następuje natychmiast prosta morfologia. Dodanie nad nią linijki *@njit(parallel=True, fastmath=True)* oraz użycie specjalnej pętli *prange* sprawia, że procesor automatycznie dzieli wiersze filmu pomiędzy wszystkie dostępne rdzenie.

```
# równoległa pętla OpenMP - rozbić wiersze obrazu na wątki CPU
for i in prange(height):
    for j in range(width):

        # równoległy filtr wygładzający
        sum_val = 0
        count = 0
        for ki in range(-half_k, half_k + 1):
            for kj in range(-half_k, half_k + 1):
                ni = i + ki
                nj = j + kj
                if ni >= 0 and ni < height and nj >= 0 and nj < width:
                    sum_val += gray_curr[ni, nj]
                    count += 1
        blurred_curr[i, j] = sum_val // count

        # równoległe różnicowanie klatek i progowanie
        pixel_diff = abs(int(blurred_curr[i, j]) - int(gray_prev[i, j]))

        if pixel_diff > threshold_val:
            mask[i, j] = 255
        else:
            mask[i, j] = 0
```

Połączenie tych operacji w jednym miejscu dało ogromną przewagę pamięciową. W wersji sekwencyjnej każda funkcja OpenCV działała „na raty” – tworzyła w pamięci RAM nową macierz, zapisywała ją, a kolejna musiała ją od nowa odczytywać. Takie ciągłe krążenie milionów pikseli między

procesorem a pamięcią RAM bardzo mocno spowalnia program (tzw. wąskie gardło przepustowości pamięci).

W wersji z OpenMP problem znika, dzięki dwóm kluczowym zmianom w strukturze kodu:

- **Zrównoleglony mikro-potok pikselowy** – zamiast osobnych etapów, każdy wątek procesora wykonuje cały pakiet operacji na swoim małym kawałku obrazu.

```
# równoległy filtr wygładzający
sum_val = 0
count = 0
for ki in range(-half_k, half_k + 1):
    for kj in range(-half_k, half_k + 1):
        ni = i + ki
        nj = j + kj
        if ni >= 0 and ni < height and nj >= 0 and nj < width:
            sum_val += gray_curr[ni, nj]
            count += 1
    blurred_curr[i, j] = sum_val // count

# równoległe różnicowanie klatek i progowanie
pixel_diff = abs(int(blurred_curr[i, j]) - int(gray_prev[i, j]))

if pixel_diff > threshold_val:
    mask[i, j] = 255
else:
    mask[i, j] = 0
```

- **Bezpieczny powrót do trybu jednowątkowego** – gdy zrównoleglone jądro skończy pracę i wypłuje czarno-białą maskę, program wraca do standardowego trybu sekwencyjnego. Czyszczenie morfologiczne oraz rysowanie konturów celowo zostawiono na jednym wątku. Te operacje nie przetwarzają już milionów pikseli, lecz gotowe, lekkie współrzędne wektorowe. Ich zrównoleglanie na CPU nie dałoby dużego zysku, a mogłoby generować błędy synchronizacji.

```
# wywołanie równoległego potoku OpenMP
mask, gray_curr_blurred = openmp_pixel_pipeline(gray_curr_raw, gray_prev, threshold_val, blur_k)

# czyszczenie maski za pomocą klasycznej morfologii (operacje na CPU)
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, morph_kernel_size)
opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
clean_mask = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel)

# wykrywanie konturów obiektów w ruchu i rysowanie ramek
contours, _ = cv2.findContours(clean_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
```

4.5 Implementacja MPI

Wariant rozproszony oparty na standardzie MPI (ang. Message Passing Interface) w pełni wykorzystuje algorytmiczną bazę z wersji sekwencyjnej. Ponieważ nasza metoda detekcji ruchu opiera się na porównywaniu dwóch sąsiednich klatek (Frame-to-Frame) i nie potrzebuje pamiętać całej

historii wideo, film możemy bez problemu pociąć na niezależne części. Zastosowaliśmy tutaj tzw. dekompozycję domenową w czasie, realizując kluczowy wymóg projektu - rozdział klatek między procesy. Zamiast obciążać jeden proces przetwarzaniem całego filmu od początku do końca, dzielimy łączną liczbę klatek przez liczbę uruchomionych procesów, dzięki czemu każdy rdzeń dostaje do obróbki swój własny, równy fragment wideo.

Wdrożenie standardu MPI wymagało przebudowy komunikacji programu oraz synchronizacji na kilku kluczowych etapach:

- **Bezpieczne pobieranie nazwy pliku i dystrybucja zadań (Broadcast)** – w celu uniknięcia sytuacji, w której każdy proces próbuje w tym samym momencie wymusić od użytkownika wpisanie tekstu w konsoli, zadanie interakcji z użytkownikiem przypisano wyłącznie procesowi głównemu ($rank == 0$). Po wpisaniu nazwy pliku, proces główny za pomocą funkcji `comm.bcast` rozgłasza ją do wszystkich pozostałych procesów roboczych. Dzięki temu każdy z nich bezpiecznie odbiera tę samą nazwę i wie, jaki plik ma otworzyć.

```
# zmienna na nazwę pliku wideo
video_name = None

# tylko proces głowy pobiera nazwę od użytkownika (aby każdy z procesów nie próbował przyjmować nazwy)
if rank == 0:
    print("--- POTOK DETEKCJI RUCHU: WARIANT ROZPROSZONY MPI ---")
    video_name = input("Podaj nazwę pliku wideo (np. video.mp4): ").strip()

# broadcast nazwy pliku do wszystkich procesów
video_name = comm.bcast(video_name, root=0)

# ścieżka do pliku
video_path = os.path.join(input_dir, video_name)
```

- **Podział pracy pomiędzy procesy MPI** - każdy proces na podstawie swojego numeru ($rank$) samodzielnie oblicza przypisany mu zakres klatek. Następnie za pomocą metody `cap.set` indywidualnie przewija wideo do swojej pozycji startowej. Nie ma tu centralnego przesyłania surowych klatek przez sieć na starcie – każdy proces pobiera dane bezpośrednio z pliku, co całkowicie likwiduje korki komunikacyjne.

```
# podział pracy pomiędzy procesy MPI
# obliczanie ile klatek przypada na każdy proces
frames_per_process = total_frames // size
start_frame = rank * frames_per_process
# ostatni proces przejmuje ewentualną resztę z dzielenia
end_frame = total_frames if rank == size - 1 else (rank + 1) * frames_per_process
num_frames_to_process = end_frame - start_frame

# każdy proces przewija wideo do swojej klatki startowej
cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
```

- **Inicjalizacja tła bazowego** - aby detekcja ruchu Frame-to-Frame działała poprawnie od pierwszej klatki przydzielonego pakietu, procesy wykonują operację cofnięcia się o jedną klatkę wstecz ($start_frame - 1$). Wczytują ją i rozmywają lokalnie, tworząc niezależne tło startowe. Dzięki temu każdy proces ma swój własny, poprawny punkt odniesienia do obliczenia różnicy *absdiff*.

```
# pobranie poprzedzającej klatki od startowej do detekcji ruchu Frame-to-Frame
if start_frame > 0:
    cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame - 1)
    ret, prev_frame = cap.read()
    gray_prev = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
    gray_prev = cv2.GaussianBlur(gray_prev, blur_kernel_size, 0)
else:
    # jeśli początek filmu, inicjalizujemy pierwszą klatką
    ret, prev_frame = cap.read()
    gray_prev = cv2.cvtColor(prev_frame, cv2.COLOR_BGR2GRAY)
    gray_prev = cv2.GaussianBlur(gray_prev, blur_kernel_size, 0)
```

- **Gromadzenie danych i synchronizacja końcowa** - po zakończeniu pętli obliczeniowej procesy robocze za pomocą operacji zbiorczej *comm.gather* przekazywana jest do procesu głównego informacja o liczbie pomyślnie przetworzonych klatek. Bezpośrednio po tym wywoływana jest bariera synchronizacyjna *comm.Barrier()*, która wymusza, aby procesor zakończył pomiar czasu dopiero w momencie, gdy ostatni proces roboczy zamknie swój potok. Proces główny (*rank == 0*) sumuje odebrane wartości, oblicza czysty wskaźnik przepustowości (FPS) i wyświetla zbiorcze podsumowanie wydajności potoku na konsoli, z pominięciem narzutu operacji wejścia/wyjścia podsystemu dyskowego.

```
# zbieranie wyników
# wszystkie procesy wysyłają swoje gotowe pakiety klatek do procesu Rank 0
all_gathered_chunks = comm.gather(len(local_frames), root=0)

# zatrzymanie stopera dla wszystkich procesów jednocześnie (Barrier)
comm.Barrier()

# prezentacja czystej wydajności algorytmu na procesorze
if rank == 0:
    end_time = time.time()
    total_time = end_time - start_time
    total_processed_frames = sum(all_gathered_chunks)
    fps_achieved = total_processed_frames / total_time

    # podsumowanie metody
    print("\n--- PODSUMOWANIE WARIANTU ROZPROSZONEGO MPI ---")
    print(f"Czas obliczeń potoku:          {total_time:.4f} sekund")
    print(f"Lacznie przetworzone klatki: {total_processed_frames}")
    print(f"Wydajność przetwarzania:      {fps_achieved:.2f} FPS\n")
```

4.6 Implementacja CUDA

Ostatnim etapem optymalizacji systemu było przeniesienie najbardziej obciążających obliczeń na kartę graficzną (GPU). Klasyczny algorytm detekcji ruchu oparty na porównywaniu sąsiednich klatek (Frame-to-Frame) wykonuje operacje matematyczne niezależnie dla każdego piksela obrazu, co czyni go idealnym kandydatem do masowego zrównoleglenia.

Pierwotnym planem było ręczne napisanie potoku pikselowego za pomocą biblioteki numba.cuda oraz CuPy. Podczas testów w środowisku systemu Windows natrafiono jednak na poważne błędy, które uniemożliwiały kompilacje kodu w locie. Ręczne kompilowanie i konfigurowanie całego środowiska C++ pod Pythonem na systemie Windows jest problematyczne i podatne na awarie.

Aby zapewnić pełną stabilność programu i sprostanie założeniom projektu, zdecydowano się na ominięcie kompilatorów i wdrożenie technologii OpenCV Transparent API (T-API), realizując wymóg projektowy CUDA/OpenCL. Architektura ta wykorzystuje fabrycznie prekompilowane jądra obliczeniowe zintegrowane bezpośrednio ze sterownikami graficznymi producenta.

Ważna uwaga: Pomimo tego, że w kodzie nie piszemy ręcznie niskopoziomowych funkcji, to pod maską w dalszym ciągu jest to czysta technologia CUDA. Na komputerach wyposażonych w karty graficzne NVIDIA, OpenCV automatycznie podpiną się pod sterownik i biblioteki uruchomieniowe CUDA. Wywołanie operacji na obiektach akcelеровanych powoduje fizyczne wysłanie danych do pamięci VRAM i uruchomienie zoptymalizowanego kodu bezpośrednio na tysiącach rdzeni GPU.

Wdrożenie akceleracji GPU i architektury T-API wymagało modyfikacji kodu na kluczowych etapach:

- **Inicjalizacja struktur w pamięci karty graficznej (VRAM)** - ważnym fragmentem do uruchomienia obliczeń na GPU jest rezygnacja z klasycznych macierzy *numpy* na rzecz obiektów klasy *cv2.UMat* (Universal Mat). Pierwsza klatka wideo, służąca jako model tła, zostaje opakowana w ten format, co powoduje natychmiastowe przesłanie danych z pamięci RAM komputera do pamięci karty graficznej (VRAM).

```
# klatka bazowa - konwersja do formatu UMat (automatyczny transfer do pamięci GPU)
first_frame_umat = cv2.UMat(first_frame)
gray_prev = cv2.cvtColor(first_frame_umat, cv2.COLOR_BGR2GRAY)
gray_prev = cv2.GaussianBlur(gray_prev, blur_kernel_size, 0)
```

- **Strumieniowe przetwarzanie potoku pikselowego na GPU** - wewnątrz głównej pętli programu każda nowo odczytana klatka wideo jest natychmiast konwertowana do formatu *cv2.UMat*. Od tego momentu wszystkie operacje – zmiana przestrzeni barw, rozmycie Gaussa, obliczanie różnic (*cv2.absdiff*), progowanie binarne (*cv2.threshold*) oraz filtry morfologiczne (*cv2.morphologyEx*) – wykonują się automatycznie na rdzeniach CUDA karty graficznej, całkowicie odciążając procesor CPU.

```
# konwersja klatki wejściowej do UMat, od tej pory operacje wykonują się na GPU
frame_umat = cv2.UMat(frame)

# preprocessing bieżącej klatki
gray_curr = cv2.cvtColor(frame_umat, cv2.COLOR_BGR2GRAY)
gray_curr = cv2.GaussianBlur(gray_curr, blur_kernel_size, 0)

# różnicowanie klatek od modelu tła
diff = cv2.absdiff(gray_curr, gray_prev)

# progowanie
_, mask = cv2.threshold(diff, threshold_val, 255, cv2.THRESH_BINARY)

# asymetryczne czyszczenie maski w pętli
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, morph_kernel_size)
opened = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel) # usuwanie szumu
clean_mask = cv2.morphologyEx(opened, cv2.MORPH_CLOSE, kernel) # scalanie przerwanych konturów
```

- **Synchronizacja i powrót do analizy geometrycznej** - algorytmy wyszukiwania konturów (`cv2.findContours`) bazują na strukturach wektorowych, operacja ta wykonuje się efektywniej na procesorze. Za pomocą metody `.get()` oczyszczona maska binarna jest ściągana z pamięci VRAM z powrotem do pamięci RAM komputera (Host). CPU przejmuje sterowanie, filtruje obiekty o zbyt małym polu powierzchni i nanosi czerwone ramki na klatkę wynikową.

```
# pobieranie maski z powrotem do pamięci CPU tylko dla operacji szukania konturów
clean_mask_cpu = clean_mask.get()

# wykrywanie konturów obiektów w ruchu i rysowanie ramek
contours, _ = cv2.findContours(clean_mask_cpu, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
result_frame = frame.copy()

for contour in contours:
    if cv2.contourArea(contour) < min_contour_area:
        continue
    # wyznaczenie boxa dla poruszającego się obiektu
    (x, y, w, h) = cv2.boundingRect(contour)
    cv2.rectangle(result_frame, (x, y), (x + w, y + h), (0, 0, 255), 2)

# aktualizacja klatki - bieżąca będzie poprzednią bezpośrednio w pamięci akcelеровanej
gray_prev = gray_curr
```

Podsumowując, użyto technologii CUDA w trochę inny sposób - nie bezpośrednio w kodzie. Przejście na OpenCV Transparent API pozwoliło na uporanie się z błędami dotyczącymi tej implementacji, jednocześnie zachowując pewność działania z wykorzystaniem tej technologii, ale w trochę inny sposób. Zamiast pisać od zera własne instrukcje dla karty graficznej, użyto gotowych, fabrycznych rozwiązań ukrytych wewnątrz biblioteki OpenCV.

Technologia CUDA użyta została na konkretne sposoby:

- **Pamięć pod stałą kontrolą GPU** - przekształcenie zwykłego obrazu w obiekt typu `cv2.UMat` działało jak bilet w jedną stronę dla naszych danych. Zamiast obciążać tradycyjną pamięć RAM komputera, film był fizycznie wysyłany bezpośrednio do pamięci VRAM karty graficznej.

- **Automatyczne uruchamianie rdzeni CUDA** - każdy krok potoku pikselowego nie angażował procesora (CPU). OpenCV automatycznie wydawało polecenia karcie graficznej NVIDIA, która do wykonania tych operacji uruchamiała swoje fabryczne, idealnie zoptymalizowane jądra obliczeniowe CUDA. Tysiące małych rdzeni na karcie graficznej przetwarzały miliony pikseli jednocześnie.

- **Wyniki** - dopiero, kiedy karta graficzna za pomocą CUDA wykonała całą pracę na pikselach i stworzyła czystą maskę ruchu, gotowy wynik był ściągany z powrotem na procesor w celu narysowania ramek. Przetworzona klatka trafia do listy w pamięci RAM, a jej fizyczny zapis na dysk odbywa się po zatrzymaniu stopera.

5. Metodologia testów wydajności

5.1 Dane Testowe

W celu dokładnej oceny sprawności systemu oraz konfiguracji i optymalizacji naszego algorytmu, przetestowano go na bazie kilku plików o różnych parametrach. Przygotowany zestaw danych testowych składał się z 4 plików, które różniły się rozdzielczością, dynamiką ruchu, specyfiką obiektów oraz czasem trwania. Pozwoliło to na weryfikację elastyczności programu w różnych warunkach sprzętowych - szczególnie w dobranych parametrach.

Szczegółowa charakterystyka wszystkich plików testowych:

- Plik **traffic.mp4**:

- **Specyfika video**: Nagranie ulicznego ruchu drogowego na trzypasmowej drodze jednokierunkowej o średnim natężeniu. Jest to plik o największej wadze i najdłuższym czasie trwania w całym zestawieniu. Ze względu na kilka poruszających się obiektów naraz, plik stanowi główne wyzwanie obliczeniowe dla testowanych architektur równoległych.

- **Rozdzielczość**: 1920 x 1080 (Full HD)

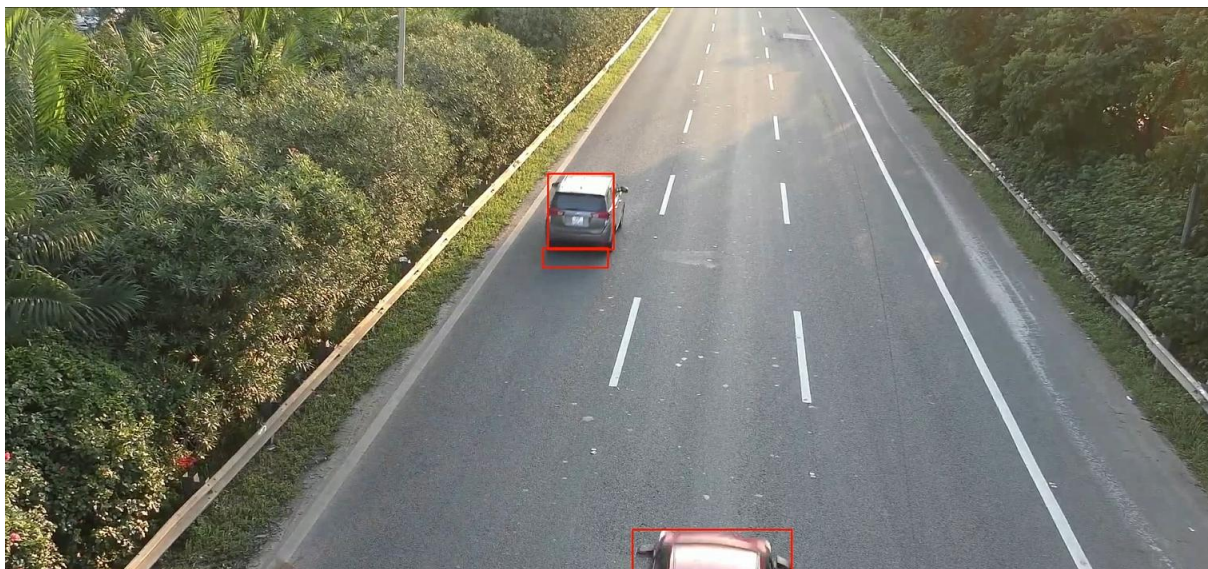
- **Liczba klatek**: 1140

- **Liczba klatek na sekundę (FPS)**: 30 klatek na sekundę

- **Czas trwania**: 38 sekund

- **Rozmiar pliku**: 196 MB

- Fragment **przykładowej klatki**:

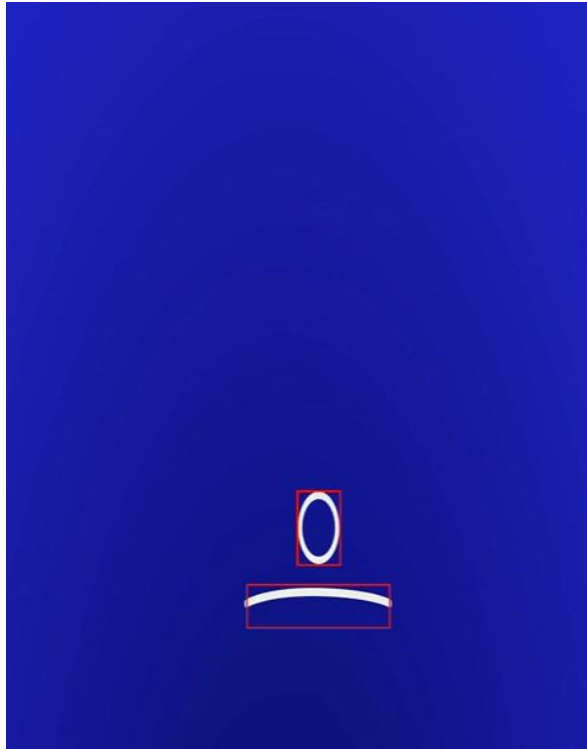


- Plik **ball.mp4**:

- **Specyfika video**: Nagranie przedstawia prostą i powtarzającą się animację odbijającej się piłki od sprężystej powierzchni na jednolitym tle. Nadaje się idealnie jako weryfikacja prostych filmików.

- **Rozdzielczość**: 1920 x 1080 (Full HD)

- **Liczba klatek:** 121
- **Liczba klatek na sekundę (FPS):** 23,98 sekund na sekundę
- **Czas trwania:** 5 sekund
- **Rozmiar pliku:** 966 KB
- Fragment **przykładowej klatki:**



- **Plik football.mp4:**

- **Specyfika wideo:** Nagranie przedstawia piłkarza oddającego strzał na bramkę. Charakteryzuje się wysoką płynnością w porównaniu do pozostałych materiałów, co przy gwałtownym ruchu obiektu wymaga od algorytmu precyzyjnego dobierania masek w celu uniknięcia rozmycia konturów.

- **Rozdzielczość:** 1920 x 1080 (Full HD)
- **Liczba klatek:** 250
- **Liczba klatek na sekundę (FPS):** 59,94 klatek na sekundę
- **Czas trwania:** 4 sekundy
- **Rozmiar pliku:** 18,7 MB
- Fragment **przykładowej klatki:**



- Plik **goal.mp4**:

- **Specyfika wideo**: Nagranie przedstawia piłkarza oddającego strzał na bramkę. Charakteryzuje się jednak niższą rozdzielczością i wiele gorszą jakością nagrania. Wykorzystanie tego pliku pozwala sprawdzić, jak automatyczne dobieranie parametrów progowania binarnego i filtracji w kodzie radzi sobie z mniejszą siatką pikseli.

- **Rozdzielczość**: 640 x 390 (Niższa jakość)
- **Liczba klatek**: 110
- **Liczba klatek na sekundę (FPS)**: 29,97 klatek na sekundę
- **Czas trwania**: 3 sekundy
- **Rozmiar pliku**: 727 KB
- Fragment **przykładowej klatki**:



5.2 Wybór analizowanych plików

Do przeprowadzenia testów wybrano trzy warianty wideo używane podczas testów programów:

- **traffic.mp4** - jako reprezentant długiego wideo testowego. Ze względu na dużą liczbę pikseli do przetworzenia pozwala zbadać rzeczywistą moc obliczeniową, skalowalność procesów rozproszonych oraz pełny potencjał akceleracji GPU. Test ten pokazuje prawdziwą wydajność, opłacalność optymalizacji i skalowalność. Dopiero na tak długim filmie narzut startowy przestaje mieć znaczenie, a zaczyna liczyć się czysta moc obliczeniowa.
- **ball.mp4** - jako reprezentant krótkiego wideo testowego w wysokiej rozdzielczości. Pozwala zbadać narzut czasowy związany z samą alokacją struktur równoległych w pamięci (np. inicjalizacja środowiska MPI czy wątków OpenMP) przy małej liczbie klatek. Test pokazuje, że uruchomienie systemów równoległych kosztuje. Zanim karta graficzna (CUDA) albo procesy (MPI) zaczną liczyć piksele, system musi zarezerwować dla nich pamięć, uruchomić wątki i podzielić zadania.
- **goal.mp4** - jako reprezentant krótkiego wideo testowego w niskiej rozdzielczości. Pozwala zbadać narzut czasowy związany z przetwarzaniem krótkiego, dynamicznego nagrania w słabej jakości. Wykorzystanie tego pliku pozwala sprawdzić, jak zaimplementowany mechanizm automatycznego, adaptacyjnego dobierania parametrów progowania binarnego i filtracji Gaussa/morfologicznej radzi sobie z mniejszą siatką pikseli oraz szumem kompresji o niskiej jakości.

5.3 Parametry wersji programów

W celu zapewnienia powtarzalności i rzetelności wyników, wszystkie eksperymenty wydajnościowe przeprowadzono według zdefiniowanego scenariusza testowego. Każdy pomiar dla danej konfiguracji sprzętowo-programowej wykonywano pięciokrotnie, a do ostatecznych zestawień i analiz

wykorzystano wyliczoną wartość średnią, co pozwoliło wyeliminować losowe skoki obciążenia systemu operacyjnego.

Konfiguracja:

- Wersja sekwencyjna – uruchomienie jednowątkowe na procesorze (CPU), wykorzystujące tylko 1 procesor logiczny. Stanowi bazowy punkt odniesienia dla obliczania przyspieszenia.
- Wersja OpenMP – potok przetwarzania zrównoleglony automatycznie na poziomie pętli przez kompilator JIT (Numba), wykorzystujący moc kolejno 2,4,8,12 wątków logicznych procesora.
- Wersja MPI – środowisko procesów rozproszonych, w którym każdy proces mapowany jest na zasoby sprzętowe CPU. W celu zbadania skalowalności w odniesieniu do architektury procesora (6 rdzeni fizycznych), testy uruchamiano kaskadowo dla 2, 4, 6 oraz 12 procesów.
- Wersja CUDA - akceleracja sprzętowa realizowana bezpośrednio w pamięci VRAM procesora graficznego (GPU), gdzie siatka wątków i bloków CUDA jest automatycznie mapowana przez mechanizm Transparent API sterownika NVIDIA.

5.4 Parametry algorytmu

Dla każdego programu, testy wydajnościowe wykonywano z przypadającym mu ze względu na rozdzielczość doбором parametrów.

Konfiguracja dla rozdzielczości 1920x1080 (pliki traffic.mp4 oraz ball.mp4) :

- **Próg detekcji ruchu (Threshold)** – 20 (wartość odcinająca szum od realnego ruchu)
- **Filtracja wygładzająca (Blur)** – filtr o rozmiarze 7x7
- **Operacje morfologiczne (Czyszczenie masek)** – dwuetapowy filtr (Otwarcie + Zamknięcie) o rozmiarze 7x7
- **Minimalne pole powierzchni konturu (Czułość)** – 300 pikseli (obiekty mniejsze jako szum tła)
- **Model tła** – algorytm Frame-to-Frame, dynamiczna aktualizacja tła, gdzie klatka bieżąca staje się odniesieniem do następnej klatki

Konfiguracja dla rozdzielczości 640x390 (plik goal.mp4) :

- **Próg detekcji ruchu (Threshold)** – 15 (wartość odcinająca szum od realnego ruchu)
- **Filtracja wygładzająca (Blur)** – filtr o rozmiarze 3x3
- **Operacje morfologiczne (Czyszczenie masek)** – dwuetapowy filtr (Otwarcie + Zamknięcie) o rozmiarze 3x3
- **Minimalne pole powierzchni konturu (Czułość)** – 50 pikseli (obiekty mniejsze jako szum tła)
- **Model tła** – algorytm Frame-to-Frame, dynamiczna aktualizacja tła, gdzie klatka bieżąca staje się odniesieniem do następnej klatki

5.5 Przebieg testów

Testy wydajnościowe zaimplementowanego potoku detekcji obiektów przeprowadzono według następujących reguł pomiarowych:

- **Powtarzalność wyników** – każdy test dla wszystkich architektur i konfiguracji rdzeni wykonano w serii 5 niezależnych uruchomień. Prezentowane wyniki stanowią średnią arytmetyczną z tych prób.
- **Rozgrzewanie procesora** – każda seria pomiarowa została poprzedzona jednym pełnym uruchomieniem testowym, którego wynik był pomijany w wyliczanej średniej. Pozwoliło to na wstępną alokację pamięci podręcznej (cache), stabilizację taktowania CPU oraz wykonanie kosztownej, pierwszej kompilacji JIT Numba w OpenMP przed właściwym pomiarem.
- **Mierzone metryki** – mierzono czas obliczeń potoku, obejmujący wyłącznie operacje w pętli przetwarzania obrazu oraz przepustowość wyliczaną jako stosunek liczby klatek wideo do czasu ich przetwarzania.
- **Obliczane metryki** – na podstawie uzyskanych czasów obliczono przyśpieszenie ($S = T/T_p$) oraz efektywność ($E = S/p$), gdzie p to liczba wątków/procesów.

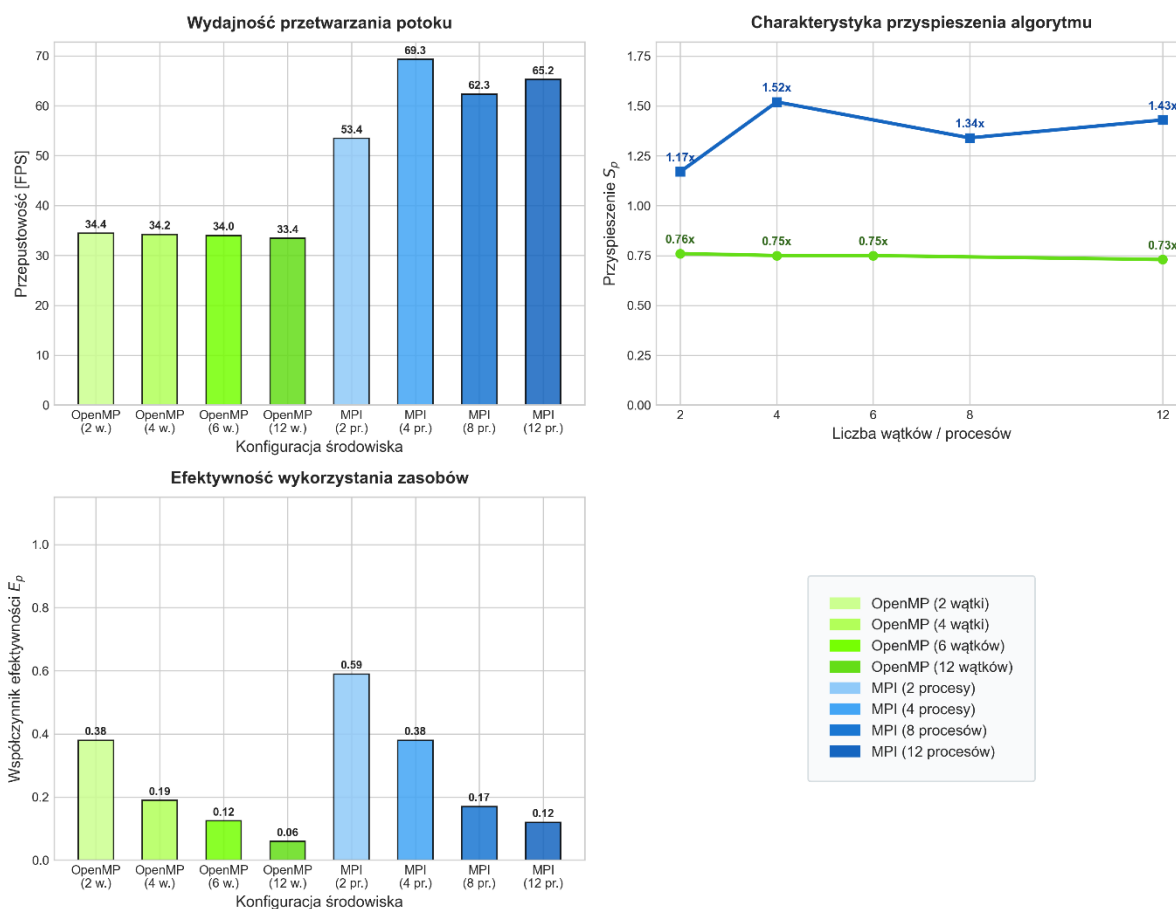
6. Wyniki

6.1 Analiza wydajności dla testu długiego (traffic.mp4)

Wariant algorytmu	Średni czas (T) [s]	Przepustowość (FPS)	Przyspieszenie (S)	Efektywność (E)
Sekwencyjny	25.0287	46.10	1.00 (baza)	1.00
OpenMP (2 wątki)	85.7167	13.29	0.29	0.15
OpenMP (4 wątki)	67.1406	16.96	0.37	0.09
OpenMP (8 wątków)	63.5730	17.92	0.39	0.05
OpenMP (12 wątków)	72.7664	15.66	0.34	0.03
MPI (2 procesy)	21.4393	53.44	1.17	0.59
MPI (4 procesy)	16.5019	69.28	1.52	0.38
MPI (6 procesów)	18.6327	62.31	1.34	0.17
MPI (12 procesów)	17.5249	65.25	1.43	0.12
CUDA/GPU (T-API)	25.6743	44.40	0.97	-

Analiza porównawcza wersji OpenMP oraz MPI dla różnej ilości wątków/procesów:

Analiza porównawcza OpenMP vs MPI dla pliku: traffic.mp4



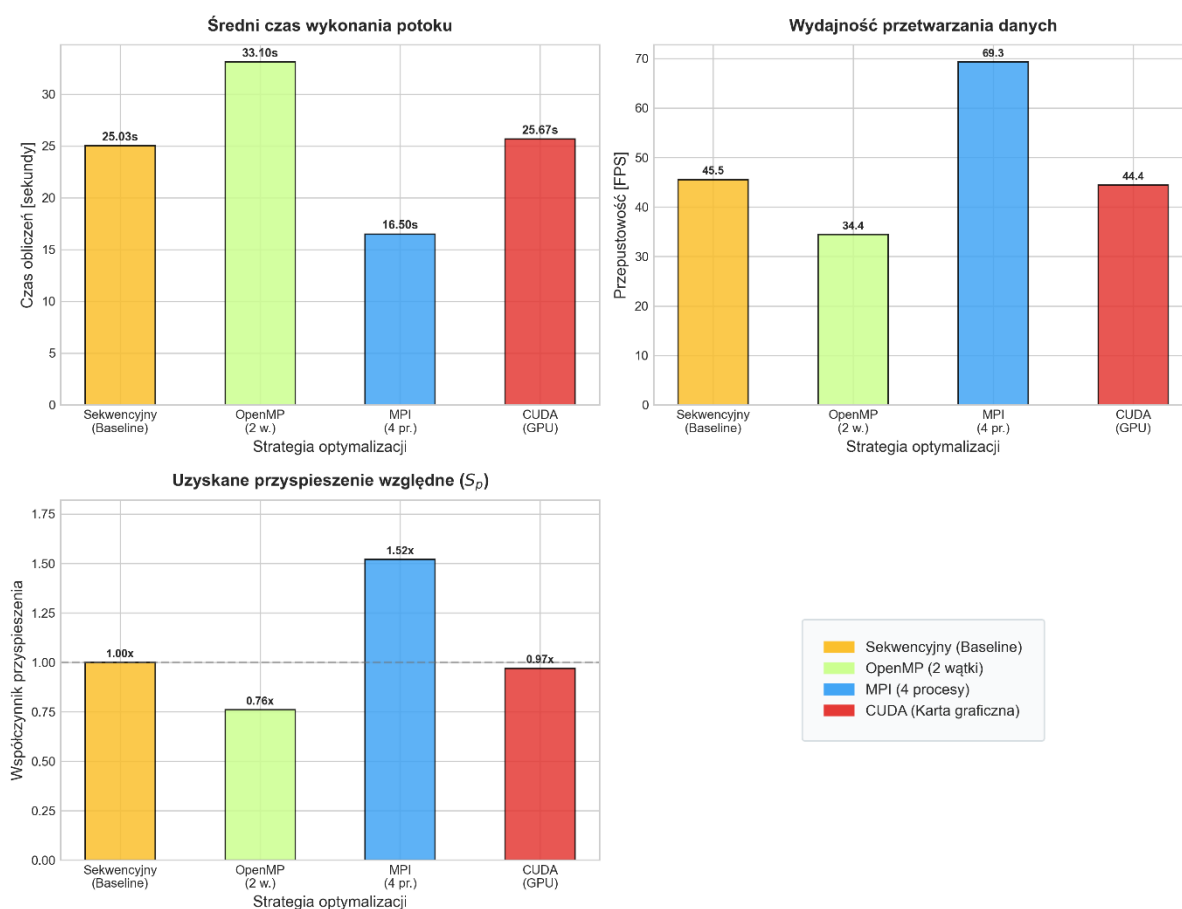
W przypadku architektury OpenMP zwiększanie liczby wątków z 2 do 12 nie przyniosło żadnych korzyści wydajnościowych, powodując stabilizację przepustowości na niskim poziomie (około 34 FPS).

Ponieważ długość filmu eliminuje wpływ rozruchu JIT Numba, winę za ten stan ponosi narzut zarządzania pamięcią współdzieloną. Najlepszym, choć mało wydajnym reprezentantem tej technologii, okazał się wariant OpenMP (2 w.).

W architekturze MPI sytuacja wygląda odwrotnie - środowisko wykazuje bardzo dobrą dynamikę skalowania, rosnąc od 53.4 FPS do szczytowych 69.3 FPS dla 4 procesów, co czyni tę konfigurację jednoznacznym zwycięzcą pojedynku. Wykresy ujawniają jednak wyraźną granicę skalowalności: powyżej 4 procesów (dla wariantów 8 i 12 pr.) wydajność zaczyna spadać, co obrazuje punkt, w którym koszty komunikacji międzyprocesowej i synchronizacji barierami dominują nad zyskiem z podziału pracy.

Analiza porównawcza końcowych wyników dla każdej wersji programu:

Zbiorcze zestawienie ostateczne dla pliku wideo: traffic.mp4



Wdrożenie optymalizacji wieloprotocowej w postaci MPI okazało się jedyną strategią, która zdołała realnie pobić wyniki wersji sekwencyjnej, skracając czas wykonania potoku z 25.03 s do 16.50 s i generując realne przyspieszenie na poziomie 1.52.

Wariant wielowątkowy OpenMP zanotował wyraźny regres wydajnościowy względem bazy sekwencyjnej, co potwierdza, że dla algorytmów OpenCV operujących na dużych rozdzielczościach Full HD paradygmat pamięci współdzielonej bez głębokiej przebudowy struktur jest nieefektywny.

Akceleracja sprzętowa CUDA wygenerowała minimalną stratę w porównaniu do wersji sekwencyjnej (spadek z 45.55 FPS do 44.40 FPS), co udowadnia istnienie tzw. wąskiego gardła magistrali PCIe (PCIe

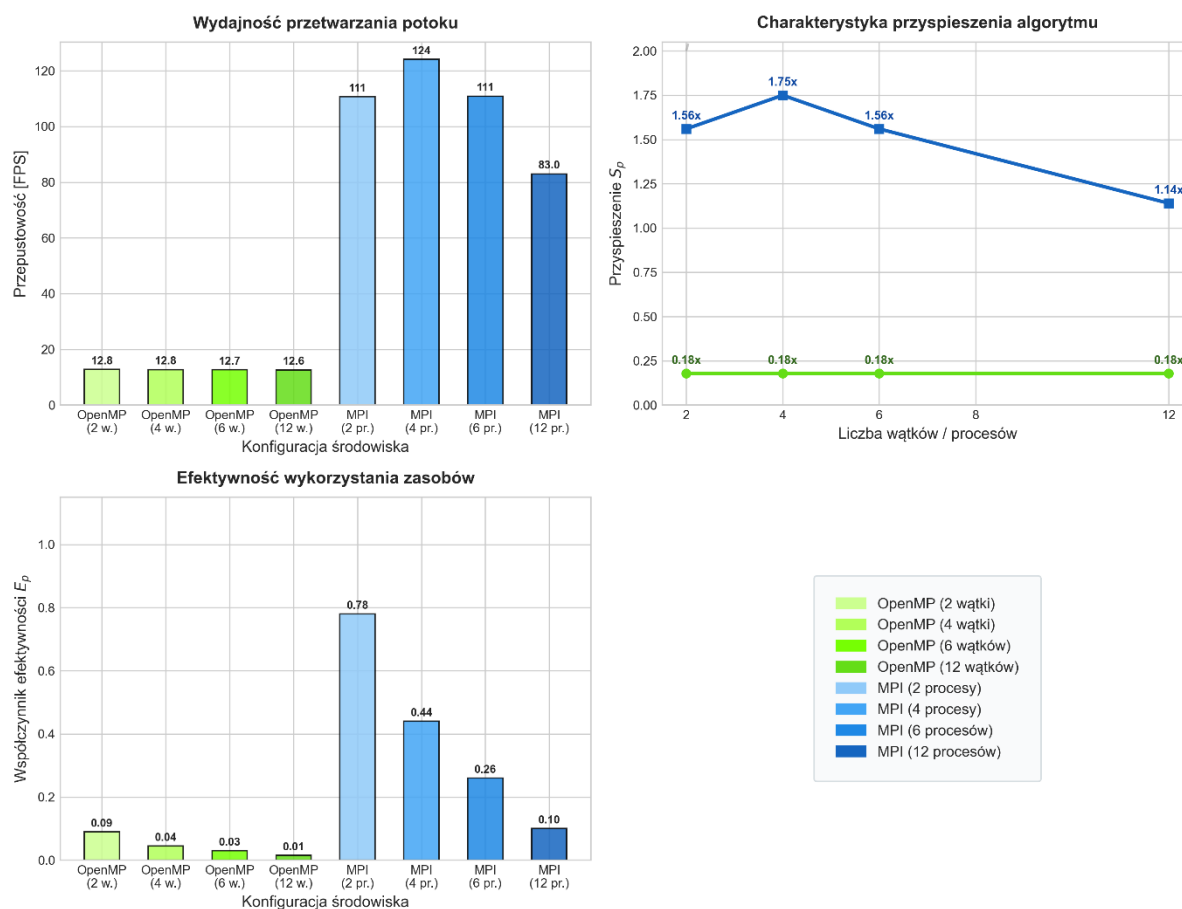
bottleneck) przy prostych potokach wizyjnych. Na nieco dłuższym filmie jednak ta strata jest znikoma w porównaniu do innych testów, co może sugerować lepsze wyniki dla jeszcze dłuższych filmów.

6.2 Analiza wydajności dla testu krótkiego (ball.mp4)

Wariant algorytmu	Średni czas (T) [s]	Przepustowość (FPS)	Przyspieszenie (S)	Efektywność (E)
Sekwencyjny	1.7030	71.05	1.00 (baza)	1.00
OpenMP (2 wątki)	9.4463	12.70	0.18	0.09
OpenMP (4 wątki)	7.2732	16.50	0.23	0.06
OpenMP (8 wątków)	6.7028	17.91	0.25	0.03
OpenMP (12 wątków)	7.6263	15.79	0.22	0.02
MPI (2 procesy)	1.0938	110.70	1.56	0.78
MPI (4 procesy)	0.9754	124.07	1.75	0.44
MPI (6 procesów)	1.0946	110.90	1.56	0.26
MPI (12 procesów)	1.4974	83.01	1.14	0.10
CUDA/GPU (T-API)	1.9966	60.15	0.85	-

Analiza porównawcza wersji OpenMP oraz MPI dla różnej ilości wątków/procesów:

Analiza porównawcza OpenMP vs MPI dla pliku: ball.mp4



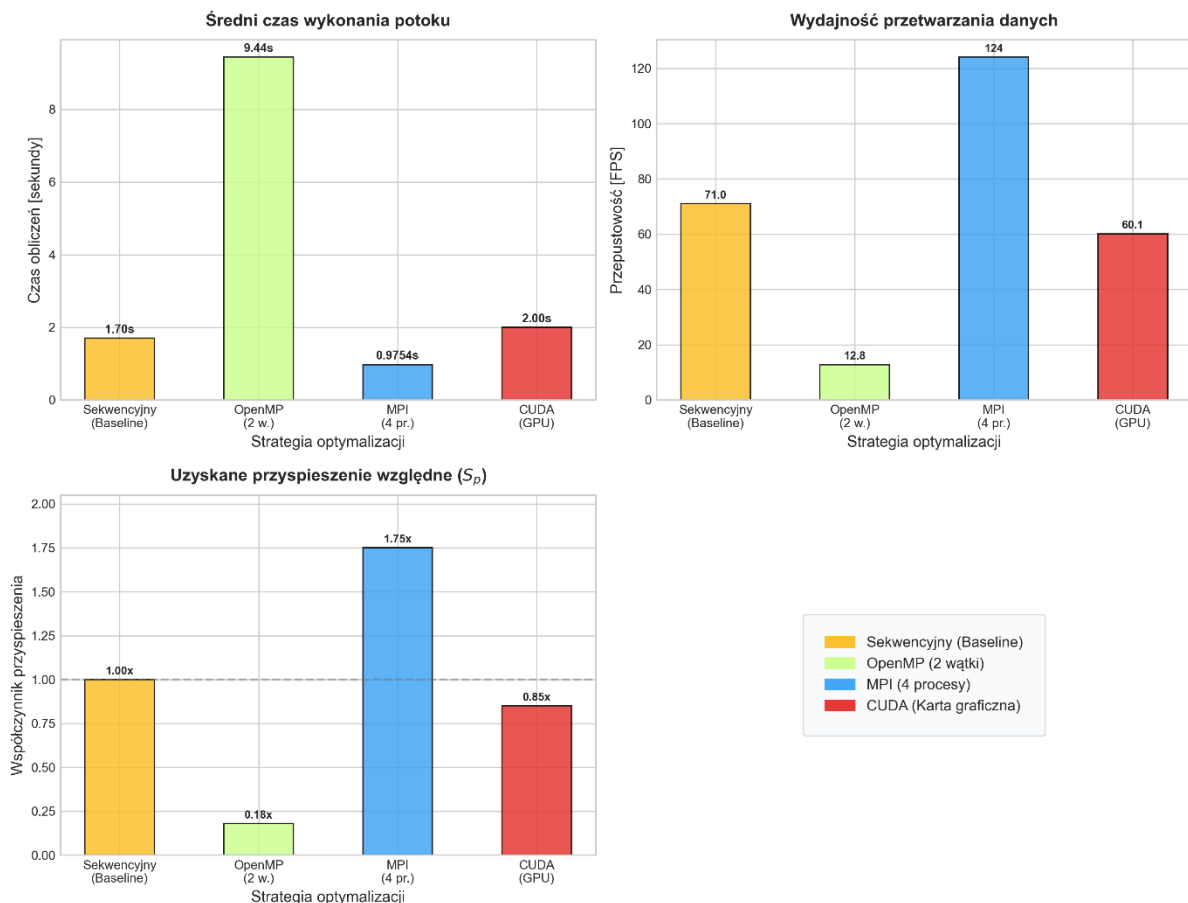
Porównując wersje algorytmu OpenMP odnotowano całkowicie płaski wykres wydajności, na którym niezależnie od ilości wątków – liczba FPS oscyluje na poziomie około 12.7. Wynik ten jest

bezpośrednim skutkiem ubocznym mechanizmu kompilacji Just-In-Time (JIT) biblioteki Numba, która przy pierwszym uruchomieniu pętli potrzebuje około 8 sekund na translację kodu do języka maszynowego. Ponieważ badany film ma zaledwie 121 klatek, etap właściwych obliczeń trwał moment, a kosztowny rozruch środowiska wielowątkowego całkowicie zdominował profil czasowy testu, czyniąc wariant OpenMP (2 w.) najlepszym wyłącznie z uwagi na najniższy narzut zarządczy.

W architekturze MPI wpływ narzutu początkowego jest znacznie mniejszy, osiągając szczytową wydajność dla 4 procesów na poziomie 124.1 FPS. Izolacja pamięci procesów pozwala na natychmiastowe, równoległe przetwarzanie klatek bez blokad systemowych, co przy mniejszej rozdzielczości pliku daje optymalny balans wydajnościowy. Wariant 2 procesowy osiąga lepsze wyniki pod kątem efektywności i przyspieszenia. Do finalnego porównania zakwalifikowano jednak wersję z 4 procesami ze względu na osiąganą przepustowość.

Analiza porównawcza końcowych wyników dla każdej wersji programu:

Zbiorcze zestawienie ostateczne dla pliku wideo: ball.mp4



Wariant wielowątkowy OpenMP ponownie zaliczył znaczny wzrost czasu obliczeń, w przypadku krótkich serii danych wejściowych narzut na kompilację rozruchową JIT Numba dyskwalifikuje to podejście.

Wdrożenie optymalizacji wieloprocessowej w konfiguracji MP4 pozwoliło na skrócenie czasu wykonania potoku wizyjnego poniżej jednej sekundy w średnim czasie – wynik 0.9754s. Zapewniło tym samym jedyne przyspieszenie względem sekwencyjnej wersji.

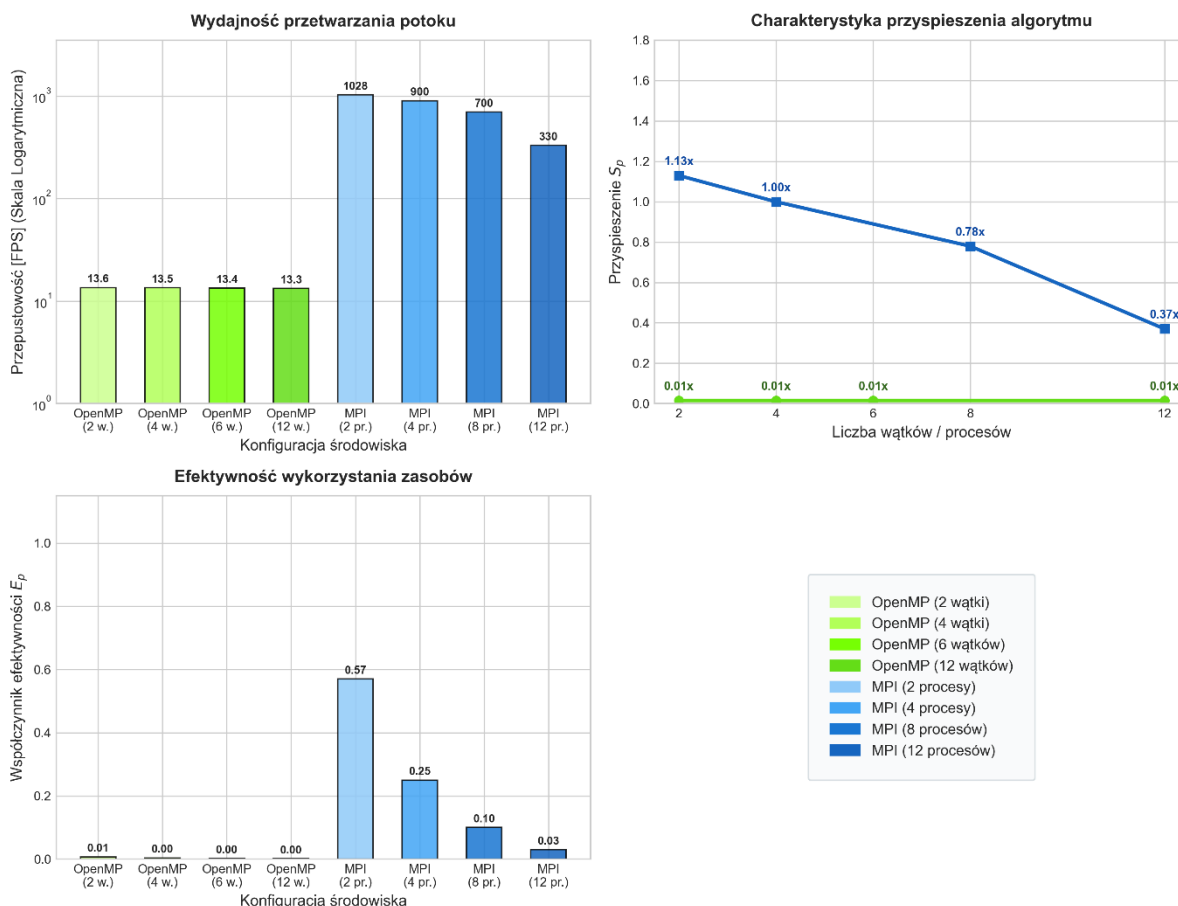
Akceleracja sprzętowa CUDA w tym scenariuszu pogorszyła czas wykonania, co przełożyło się na spadek przepustowości do poziomu 60.1 FPS i ujemny współczynnik przyspieszenia. Spadek ten potwierdza negatywny wpływ wąskiego gardła magistrali PCIe, który w przypadku krótszego wideo jest bardziej odczuwalny – czas alokacji pamięci na GPU oraz transferów klatek Host-Device drastycznie przewyższa czas czystych obliczeń graficznych.

6.3 Analiza wydajności dla testu niskiej jakości (goal.mp4)

Wariant algorytmu	Średni czas (T) [s]	Przepustowość (FPS)	Przyspieszenie (S)	Efektywność (E)
Sekwencyjny	0.1235	883.86	1.00 (baza)	1.00
OpenMP (2 wątki)	2.1141	51.84	0.06	0.03
OpenMP (4 wątki)	1.9919	54.76	0.06	0.02
OpenMP (8 wątków)	1.9171	56.90	0.06	0.01
OpenMP (12 wątków)	2.0204	52.85	0.06	0.01
MPI (2 procesy)	0.1090	1028.24	1.13	0.57
MPI (4 procesy)	0.1230	899.95	1.00	0.25
MPI (6 procesów)	0.1588	699.89	0.78	0.10
MPI (12 procesów)	0.3349	329.75	0.37	0.03
CUDA/GPU (T-API)	0.2241	487.89	0.55	-

Analiza porównawcza wersji OpenMP oraz MPI dla różnej ilości wątków/procesów:

Analiza porównawcza OpenMP vs MPI dla pliku: goal.mp4

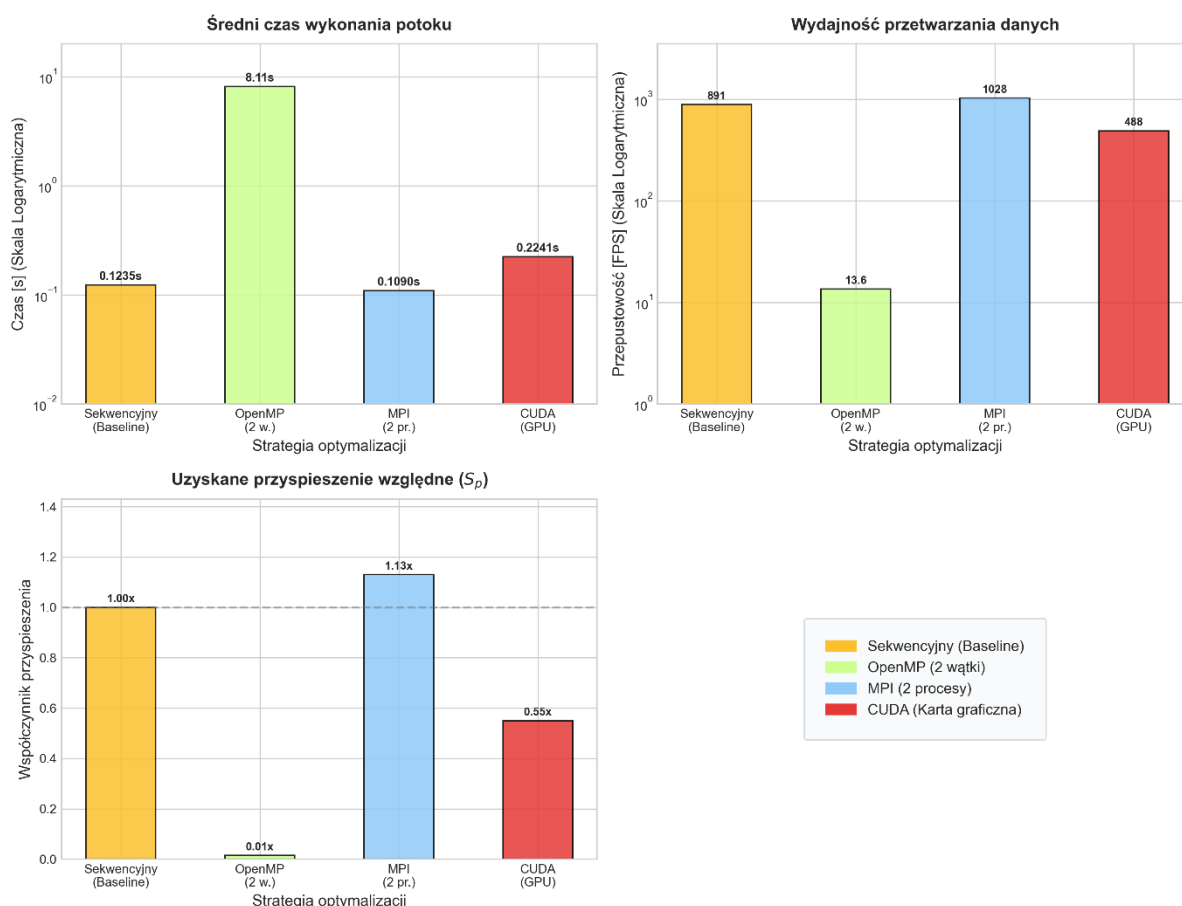


W architekturze OpenMP przetwarzanie niskiej jakości wideo o krótkim czasie trwania ukazało całkowitą nieefektywność w dodawaniu kolejnych wątków przy płaskiej przepustowości i spadającej efektywności. Przyczyną tak potężnego regresu jest niezmiennie narzut rozruchowy kompilacji JIT Numba, który przy czasie właściwych obliczeń rzędu milisekund całkowicie paraliżuje działanie programu. Wariant OpenMP z 2 wątkami uznano za najlepszy z najgorszych.

W architekturze MPI, z uwagi na brak konieczności kompilacji w locie, środowisko zachowało pełną responsywność, a trend na wykresie przyspieszenia ujawnił podręcznikowy przykład działania prawa Amdahla. Najwyższą wydajność, a zarazem zwycięstwo w całym pojedynku, odnotowano dla wariantu MPI (2 procesy), który osiągnął rekordową przepustowość 1028.24 FPS oraz efektywność na poziomie 0.57. Każde dalsze skalowanie środowiska w górę (4, 8 i 12 procesów) drastycznie pogarszało wyniki, ponieważ dla tak małego strumienia danych narzut systemowy tworzenia i barierowej synchronizacji procesów błyskawicznie przewyższył czas potrzebny na faktyczną detekcję obiektów.

Analiza porównawcza końcowych wyników dla każdej wersji programu:

Zbiórce zestawienie ostateczne dla pliku wideo: goal.mp4



Wielowątkowość OpenMP zanotowała znaczne wydłużenie czasu trwania programu. Dla ultrakrótkich potoków przetwarzania obrazów technologia kompilacji JIT w locie staje się skrajnie nieoptymalna.

Architektura MPI pozwoliła na optymalne zrównoleglenie potoku, dzięki czemu jako jedyne skróciło ono czas wykonania poniżej progu bazowego.

Akceleracja GPU przy użyciu wariantu CUDA ponownie pogorszyła wynik sekwencyjnej wersji.

6.4 Podsumowanie wyników i najważniejsze wnioski

Eksperymenty wydajnościowe przeprowadzone na różnych wideo pozwoliły na zmapowanie unikalnych cech, ograniczeń oraz granic opłacalności każdego algorytmu i pozwoliły na sformułowanie kluczowych wniosków dla projektu:

- **Dominacja wersji MPI** – środowisko wieloprocesowe MPI okazało się bezdyskusyjnie najskuteczniejszą strategią optymalizacji na wybranych filmach wideo. Dzięki pełnej izolacji przestrzeni adresowej procesów, współbieżne wywołania funkcji biblioteki OpenCV operowały na niezależnych obszarach pamięci RAM. Zapobiegło to konfliktom dostępu oraz narzutom synchronizacyjnym, które sparaliżowały konkurencyjne podejście wielowątkowe.
- **Problem narzutu JIT w OpenMP** – technologia OpenMP oparta na kompilacji w locie okazała się skrajnie nieefektywna. Stały koszt kompilacji rozruchowej zdyskwalifikował tę metodę przy krótkich plikach. Z kolei przy dłuższym teście, gdzie wpływ JIT został zamortyzowany, barierą nie do przejścia okazał się paradygmat pamięci współdzielonej. Zarządzanie wątkami alokującymi ciężkie struktury danych wewnątrz pętli generowało zbyt wysoki koszt synchronizacji.
- **Granica skalowalności** – badania pokazały istnienie punktu krytycznego dla skalowania liczby rdzeni procesora. W przypadku MPI najwyższą wydajność uzyskiwano dla konfiguracji z 2 lub 4 procesami. Przekroczenie tego pułapu (8 i 12 procesów) niezmiennie skutkowało spadkiem wydajności. Może być to bezpośrednia konsekwencja wzrostu złożoności komunikacji.
- **Ograniczenie akceleracji sprzętowej dla CUDA** – wyniki wariantu CUDA ukazały, że implementacja masowo równoległego przetwarzania na GPU ma głęboki sens wyłącznie przy wysokiej złożoności obliczeniowej samego algorytmu. W przypadku prostego potoku, krytycznym punktem programu stało się wąskie gardło magistrali PCIe. Warto jednak odnotować zbliżony wynik do wersji sekwencyjnej w teście dłuższego wideo, co może prowadzić do hipotezy o uzyskaniu lepszych wyników dla jeszcze bardziej zaawansowanych filmów, które trwały by dłużej.

7. Instrukcja obsługi i uruchomienia

7.1 Wymagania

Do poprawnego uruchomienia wszystkich wersji aplikacji oraz powtórzenia testów wydajnościowych wymagane jest spełnienie następujących warunków środowiskowych i sprzętowych:

- **System operacyjny** – Microsoft Windows 10 lub Windows 11
- **Środowisko uruchomieniowe** – Środowisko lokalne języka Python w wersji 3.10 lub nowszej (zalecana i przetestowana wersja 3.12)
- **Zarządzenie pakietami** – Narzędzie pip zintegrowane z aktywnym interpreterem języka Python
- **Środowisko rozproszone (wersja MPI)** – Zainstalowane w systemie operacyjnym oficjalne oprogramowanie Microsoft MPI (MS-MPI v10.x)
- **Procesor (CPU)** – Wielordzeniowy układ wspierający wielowątkowość, projekt zoptymalizowano pod kątem procesora oferującego 6 rdzeni fizycznych i 12 wątków logicznych
- **Karta graficzna (GPU)** – karta graficzna firmy **NVIDIA** z dedykowaną pamięcią VRAM oraz zainstalowanymi oficjalnymi sterownikami systemowymi wspierającymi architekturę masowo równoległą NVIDIA CUDA.

7.2 Uruchomienie programu

Przed przystąpieniem do uruchamiania skryptów należy przeprowadzić instalację pakietów zewnętrznych. Wszystkie operacje należy wykonywać z poziomu terminala systemowego (np. wbudowanego w środowisko edycyjne Visual Studio Code), który pełni rolę interfejsu w naszym projekcie. Uruchamiamy pliki wpisując poniższe komendy, znajdując się bezpośrednio w katalogu głównym projektu:

- **Instrukcja instalacji bibliotek** – w celu automatycznego pobrania i zainstalowania wymaganych modułów

```
python -m pip install -r requirements.txt
```

- **Struktura katalogów wejścia i wyjścia** – do zachowania poprawnej struktury plików

videos_input/ - katalog do umieszczania wejściowych filmów testowych

videos_output/ - katalog, w którym zapisywane są gotowe pliki wideo z naniesionymi czerwonymi ramkami detekcji obiektów, zawiera podkatalogi zgodne z uruchamianą wersją programu

- **Uruchamianie wersji programów na domyślnych ustawieniach** – uruchomienie każdego programu odbywa się przez wywołanie go w terminalu oraz podanie pliku wejściowego. Poproszony o to użytkownik wpisuje nazwę pliku z katalogu wejściowego razem z rozszerzeniem (np. ball.mp4) i zatwierdza, program zwraca nam wyniki testów wydajnościowych oraz zapisuje plik w katalogu wyjściowym. Uruchomienie z poniższymi komendami pozwala na przyjęcie domyślnych konfiguracji.

```
python -m sequential
```

```
python -m openmp (domyślnie dla maksymalnej ilości wątków)
```

python -m mpi

python -m cuda

- **Uruchamianie wersji rozproszonej w architekturze wielu wątków** – wywołanie wariantu OpenMP ze zdefiniowaną liczbą wątków

set NUMBA_NUM_THREADS=X && python -m openmp dla zdefiniowania ilości wątków w wersji OpenMP, gdzie X oznacza liczbę procesów

- **Uruchamianie wersji rozproszonej w architekturze wielu procesów** – wywołanie wariantu MPI ze zdefiniowaną liczbą procesów

mpiexec -n X python -m mpi dla zdefiniowania ilości procesów w wersji MPI, gdzie X oznacza liczbę procesów

Przykładowe uruchomienie programu i wynik w postaci podsumowania: zapisu pliku, czasu obliczenia oraz FPS:

```
C:\Users\MIŁOSZ\Documents\GitHub\parallel-video-motion-detection>python -m sequential
--- POTOK DETEKCJI RUCHU: WARIANT SEKWENCYJNY ---
Podaj nazwe pliku wideo (np. video.avi lub video.mp4): ball.mp4

[START] Analiza pliku: videos_input\ball.mp4
[INFO] Rozdzielczosc: 1080x1920, Klatki: 121
[INFO] Wynik zostanie zapisany w: videos_output\sequential\ball_seq.mp4
Przetwarzanie...

--- PODSUMOWANIE WARIANTU SEKWENCYJNEGO ---
Czas obliczen:          2.0081 sekund
Przetworzone klatki w petli: 120
Wydajnosc przetwarzania:  59.76 FPS
```

8. Podsumowanie projektu

Projekt poświęcony był implementacji oraz empirycznej analizie porównawczej wydajności potoku detekcji obiektów ruchomych w wideo z wykorzystaniem paradygmatów obliczeń sekwencyjnych, wielowątkowych (OpenMP), wieloprocessowych (MPI) oraz akceleracji sprzętowej (CUDA).

Badania pokazały najwyższą efektywność dla architektury z użyciem MPI. Pełna izolacja pamięci procesów eliminuje konflikty alokacji w locie i minimalizuje potrzebę niskopoziomowej synchronizacji, co pozwoliło uzyskać stabilny zysk wydajnościowy we wszystkich testach. Wykorzystanie biblioteki Numba ukazało jej ograniczenia w postaci stałego narzutu oraz zarządzania pamięcią współdzieloną.

Wyniki akceleracji sprzętowej CUDA są podstawą do wysnucia hipotezy o rosnącej jej efektywności dla bardziej zaawansowanych i dłuższych form wideo.

Realizując projekt spełniono zakładane cele, opracowano stabilne środowisko testowe, zaimplementowano odmienne podejścia i przeprowadzono ich analizę porównawczą na zróżnicowanych danych wejściowych. Dodatkowo efektem projektu staje się także sam program do detekcji ruchu, nastawiony na detekcję ruchu metodą Frame-to-Frame.